ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

RAPPORT DE PROJET

Application de gestion d'une salle de sport

Python · Tkinter · SQLite

Module	Python avancé
Établissement	ENSAM Casablanca – IAGI, 1ère année
Réalisé par	GURAINI YAHYA · GAHEM ISMAIL · QUASSAY SAAD
Encadrant	M. MUSTAPHA HAIN
Année universitaire	2026 – 2027
Date limite	le 08 novembre 2026



Sommaire

1. Introduction **2**
2. Cahier des charges **2**
2.1 Besoins fonctionnels 2
2.2 Contraintes 2
2.3 Organisation du travail 3
3. Conception de la base de données **3**
3.1 Règles de gestion 3
3.2 Dictionnaire de données 3
3.3 Modèle conceptuel de données (MCD) 4
3.4 Modèle logique de données (MLD) 4
3.5 Choix de conception 4
3.6 Script de création 4
4. Architecture logicielle **5**
5. Réalisation **6**
5.1 Interface graphique 6
5.2 Validation des saisies 7
5.3 Données enregistrées dans la base 8
6. Tests **8**
7. Difficultés rencontrées et solutions **9**
8. Conclusion et perspectives **9**

1. Introduction

La gestion d'une salle de sport repose sur trois informations liées entre elles : qui est inscrit (les adhérents), à quoi chacun a droit (les abonnements) et ce qu'il a déjà réglé (les paiements). Lorsqu'elles sont tenues sur papier ou dans un tableur, ces informations deviennent vite incohérentes : abonnement expiré non repéré, paiement oublié, doublon d'adhérent.

Ce projet répond à ce besoin par une application de bureau en Python, avec une interface graphique Tkinter et une base de données SQLite. Il respecte les contraintes du module : une base de trois tables au maximum, une application de gestion complète et un travail réalisé en groupe de trois étudiants.

Le rapport présente successivement le cahier des charges, la conception de la base de données (MCD et MLD), l'architecture logicielle, la réalisation, les tests, puis les difficultés rencontrées et les perspectives.

2. Cahier des charges

2.1 Besoins fonctionnels

Module	Fonctionnalités attendues
Adhérents	Ajouter, modifier, supprimer un adhérent ; rechercher par nom ou prénom (résultats en direct) ; contrôler le téléphone (chiffres) et l'email (forme nom@domaine.ext).
Abonnements	Associer un abonnement à un adhérent ; quatre formules (1, 3, 6 ou 12 mois) ; date de fin et prix proposés automatiquement ; statut Actif / Expire bientôt / Expiré affiché en couleur ; renouvellement en un clic.
Paiements	Choisir un abonnement ; afficher son prix, le montant déjà payé et le reste à payer ; enregistrer un paiement (Espèces, Carte Bancaire, Chèque, Virement) ; consulter l'historique.
Tableau de bord	Afficher le nombre d'adhérents, d'abonnements actifs et expirés, les revenus du mois, les abonnements qui expirent sous 7 jours et l'histogramme des revenus par mois.

2.2 Contraintes

- Langage Python 3, interface Tkinter / ttk uniquement ; seule la bibliothèque standard est utilisée (sqlite3, tkinter, datetime, calendar).
- Base de données SQLite locale, limitée à 3 tables.

- Application utilisable hors connexion, avec un seul fichier de base (gym.db) créé automatiquement au premier lancement.
- Messages d'erreur clairs en français ; confirmation avant toute suppression.

2.3 Organisation du travail

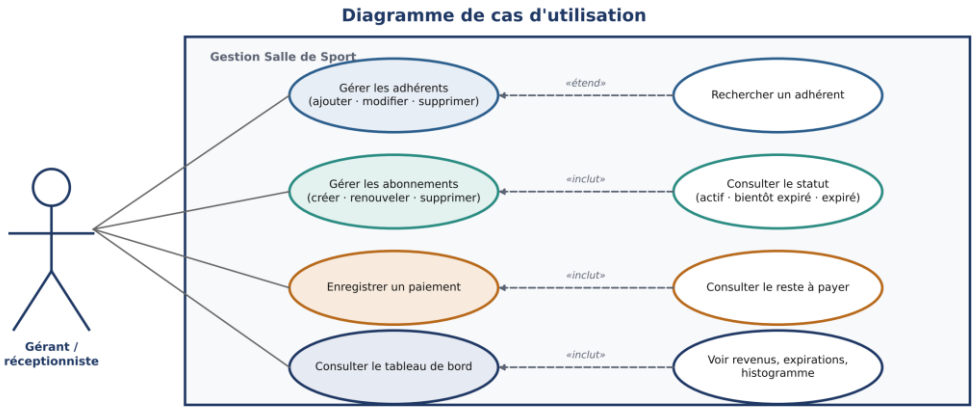


Figure 1 – cas d'utilisation : le gérant pilote les quatre modules de l'application.

3. Conception de la base de données

3.1 Règles de gestion

- **R1 –** Un adhérent peut souscrire plusieurs abonnements ; un abonnement appartient à un seul adhérent.
- **R2 –** Un abonnement peut être réglé en plusieurs paiements ; un paiement concerne un seul abonnement.
- **R3 –** La date de fin d'un abonnement est égale à la date de début plus la durée du type (1, 3, 6 ou 12 mois), en gérant les fins de mois.
- **R4 –** Supprimer un adhérent supprime ses abonnements, et donc leurs paiements (suppression en cascade).
- **R5 –** Le statut (Actif, Expire bientôt, Expiré) et le reste à payer sont calculés, jamais stockés.
- **R6 –** Un montant de paiement est strictement positif ; s'il dépasse le reste à payer, l'utilisateur doit confirmer.

3.2 Dictionnaire de données

Entité	Attribut	Type	Description / contrainte
ADHÉRENT	id	Entier	Identifiant (clé primaire, auto-incrémenté)
	nom, prenom	Texte	Obligatoires
	telephone	Texte	Facultatif ; chiffres uniquement (espaces et + tolérés)
	email	Texte	Facultatif ; contient @ et un domaine avec un point
	date_inscription	Date	Date du jour à la création (AAAA-MM-JJ)
ABONNEMENT	id	Entier	Identifiant (clé primaire, auto-incrémenté)
	adherent_id	Entier	Clé étrangère vers adherents(id) ; obligatoire
	type	Texte	1 Mois, 3 Mois, 6 Mois ou 12 Mois
	date_debut, date_fin	Date	Fin = début + durée du type
	prix	Réel	En dirhams, > 0 ; proposé selon le type (300 / 800 / 1500 / 2800)
PAIEMENT	id	Entier	Identifiant (clé primaire, auto-incrémenté)
	abonnement_id	Entier	Clé étrangère vers abonnements(id) ; obligatoire
	montant	Réel	En dirhams, strictement positif
	date_paiement	Date	Date du jour à l'enregistrement
	mode	Texte	Espèces, Carte Bancaire, Chèque ou Virement

3.3 Modèle conceptuel de données (MCD)

Le MCD suit la méthode Merise. Les deux associations, SOUSCRIRE et RÉGLER, sont de type « un à plusieurs » : le côté (1,1) désigne l'entité dépendante (l'abonnement, le paiement).

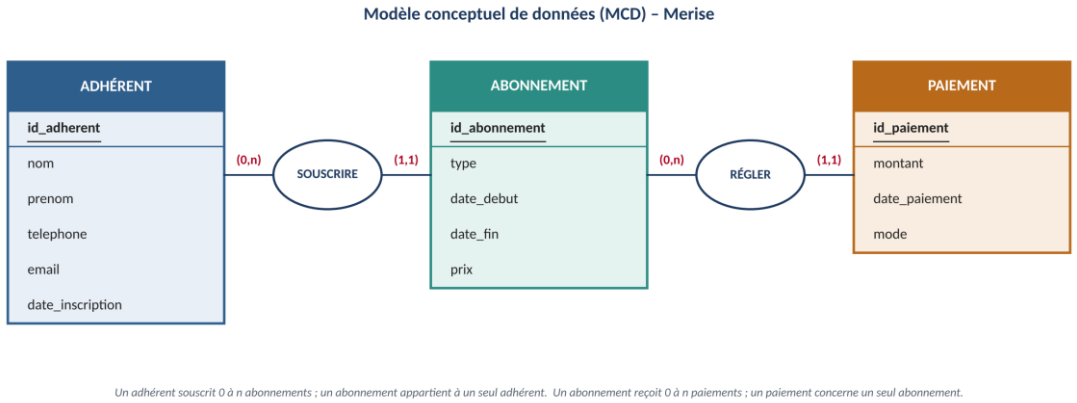


Figure 2 – modèle conceptuel de données (identifiants soulignés).

3.4 Modèle logique de données (MLD)

Pour une association 1-N, la clé primaire du côté 1 devient clé étrangère du côté N. On obtient trois relations (clé étrangère précédée de #) : **adherents** (id, nom, prenom, telephone, email, date_inscription) ; **abonnements** (id, #adherent_id, type, date_debut, date_fin, prix) ; **paiements** (id, #abonnement_id, montant, date_paiement, mode).

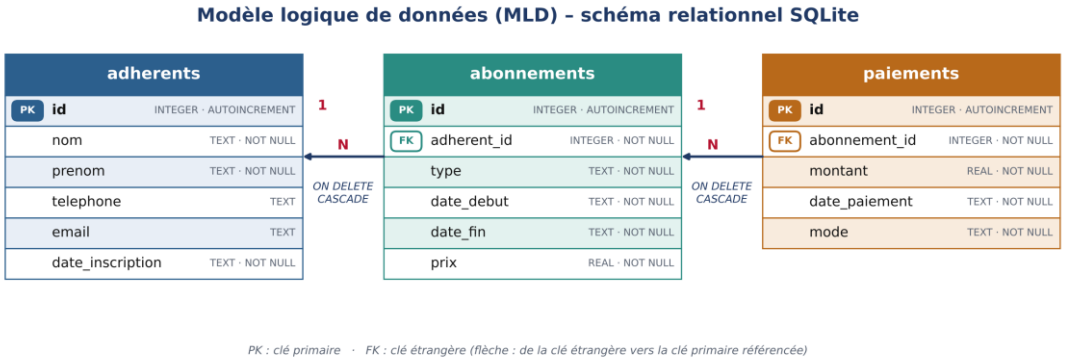


Figure 3 – modèle logique de données avec contraintes d'intégrité.

3.5 Choix de conception

- **Troisième forme normale :** chaque attribut dépend uniquement de la clé de sa table ; le nom de l'adhérent n'est pas répété dans les abonnements (on utilise adherent_id).
- **Données calculées non stockées (R5) :** le statut dépend de date_fin et de la date du jour ; le reste à payer vaut prix − somme des paiements. Les stocker créerait des risques d'incohérence.
- **Intégrité par le SGBD :** les clés étrangères avec ON DELETE CASCADE garantissent qu'aucun abonnement ni paiement ne reste orphelin. La commande PRAGMA foreign_keys = ON est exécutée à chaque connexion, sinon SQLite ignore les clés étrangères.
- **Dates au format ISO (AAAA-MM-JJ) :** le tri alphabétique est alors identique au tri chronologique, et les comparaisons SQL de dates (date_fin >= ?) fonctionnent directement.

3.6 Script de création

```
CREATE TABLE IF NOT EXISTS adherents (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL, prenom TEXT NOT NULL,
    telephone TEXT, email TEXT, date_inscription TEXT NOT NULL );
CREATE TABLE IF NOT EXISTS abonnements (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    adherent_id INTEGER NOT NULL, type TEXT NOT NULL,
    date_debut TEXT NOT NULL, date_fin TEXT NOT NULL, prix REAL NOT NULL,
    FOREIGN KEY(adherent_id) REFERENCES adherents(id) ON DELETE CASCADE );
CREATE TABLE IF NOT EXISTS paiements (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    abonnement_id INTEGER NOT NULL, montant REAL NOT NULL,
    date_paiement TEXT NOT NULL, mode TEXT NOT NULL,
    FOREIGN KEY(abonnement_id) REFERENCES abonnements(id) ON DELETE CASCADE );
```

4. Architecture logicielle

Le projet tient dans un seul fichier Python, organisé en trois couches qui ne communiquent que de haut en bas : l'interface appelle la classe Database, qui est la seule à parler à SQLite. Aucune requête SQL ne figure dans les classes d'interface.

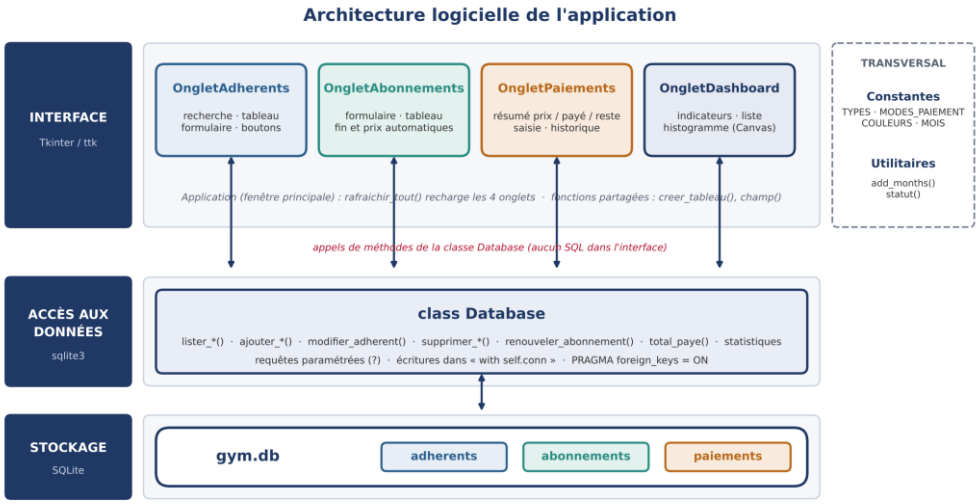


Figure 4 – architecture de l'application.

Couche	Rôle	Éléments principaux
Interface	Afficher, lire la saisie, présenter les erreurs	Application, OngletAdherents, OngletAbonnements, OngletPaiements, OngletDashboard, creer_tableau(), champ()
Accès aux données	Exécuter le SQL de façon sûre, calculer les statistiques	Classe Database : lister_*, ajouter_*, modifier_adherent, supprimer_*, renouveler_abonnement, total_paye, nb_*, revenus_*
Stockage	Conserver les données	gym.db (3 tables)
Transversal	Règles partagées	TYPES, MODES_PAIEMENT, COULEURS, add_months(), statut()

Synchronisation interface – base. L'interface n'est jamais la source de vérité : après chaque ajout, modification ou suppression, la méthode rafraichir_tout() demande à chaque onglet de se recharger depuis la base. Elle est aussi appelée à chaque changement d'onglet.

```
class Application(tk.Tk):
    def __init__(self, db):
        ...
        self.nb.bind("<<NotebookTabChanged>>", lambda e: self.rafraichir_tout())

    def rafraichir_tout(self):
        for onglet in self.onglets:       # Adhérents, Abonnements, Paiements, Tableau de bord
            onglet.rafraichir()

def ajouter(self):                         # dans OngletAdherents
    donnees = self.lire_formulaire()       # contrôles de saisie
    if donnees:
        self.db.ajouter_adherent(*donnees) # INSERT paramétré
        self.vider()
        self.app.rafraichir_tout()         # tous les onglets sont actualisés
```

Sécurité des requêtes. Toutes les requêtes sont paramétrées (marqueurs ?) : aucune valeur saisie n'est concaténée dans le SQL, ce qui écarte les injections. Les écritures sont placées dans un bloc « with self.conn », qui valide la transaction (commit) ou l'annule en cas d'erreur (rollback).

5. Réalisation

5.1 Interface graphique

La fenêtre principale contient un Notebook (onglets) : Adhérents, Abonnements, Paiements et Tableau de bord. À chaque changement d'onglet, celui-ci recharge ses données : un adhérent ajouté dans le premier onglet apparaît donc immédiatement dans la liste déroulante du deuxième.

Onglet	Composants	Comportements notables
Adhérents	Barre de recherche, tableau, formulaire à 4 champs, boutons Ajouter / Modifier / Supprimer / Vider	Clic sur une ligne : le formulaire se remplit. Suppression avec confirmation (cascade annoncée).
Abonnements	Liste d'adhérents, type, dates, prix, tableau coloré, boutons Renouveler / Supprimer	Changer le type recalcule la date de fin et le prix. Lignes vertes, orange ou rouges selon le statut.
Paiements	Choix de l'abonnement, encadré Prix / Déjà payé / Reste à payer, montant, mode, historique	L'encadré se met à jour après chaque paiement ; le montant est pré-rempli avec le reste.
Tableau de bord	4 indicateurs, liste des expirations sous 7 jours, histogramme (Canvas)	Tout est calculé par des requêtes SQL d'agrégation (COUNT, SUM, GROUP BY).

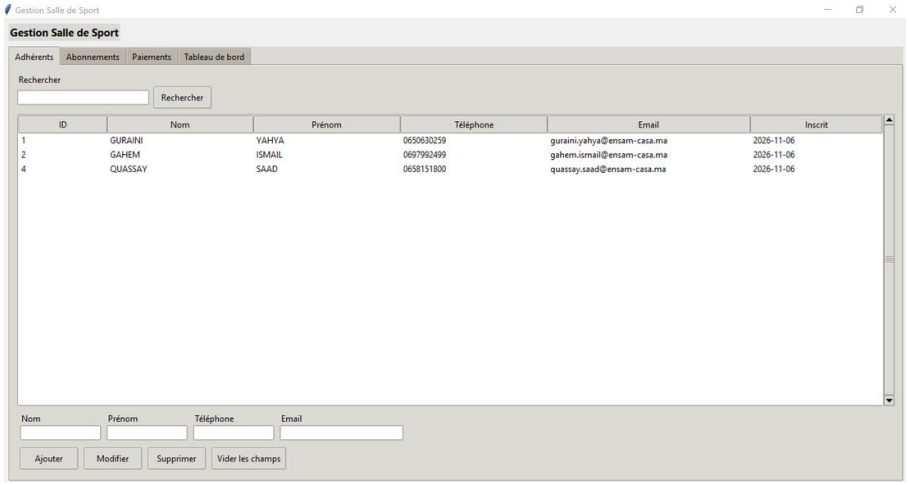


Figure 5 – onglet Adhérents : les trois membres du groupe enregistrés dans la base.

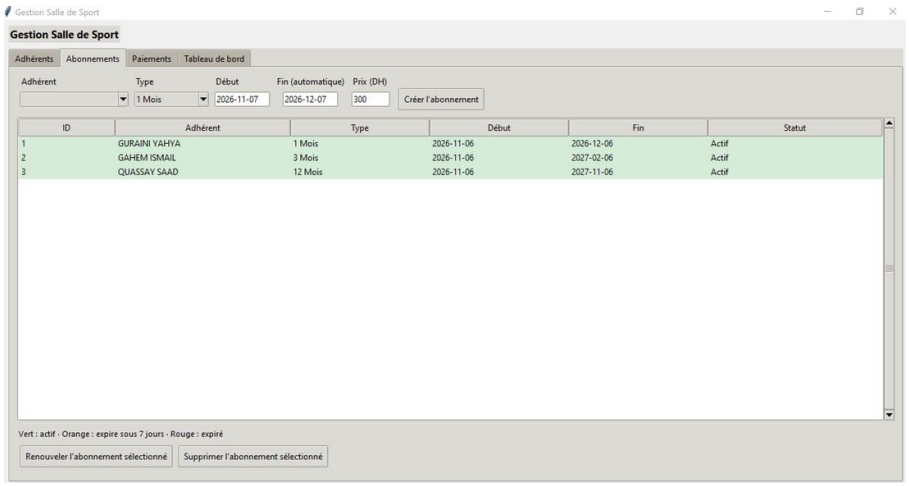


Figure 6 – onglet Abonnements : formules de 1, 3 et 12 mois, dates de fin calculées, statut Actif.

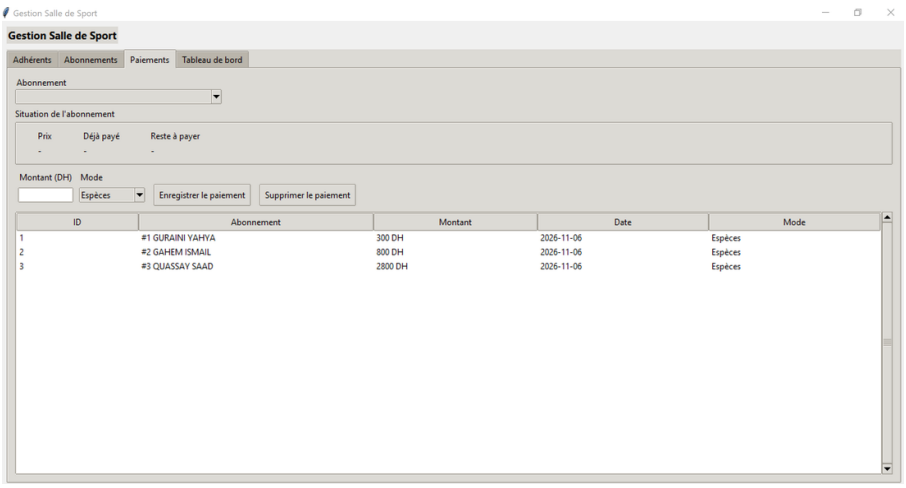


Figure 7 – onglet Paiements : historique des trois paiements ; l'encadré se remplit dès qu'un abonnement est choisi.

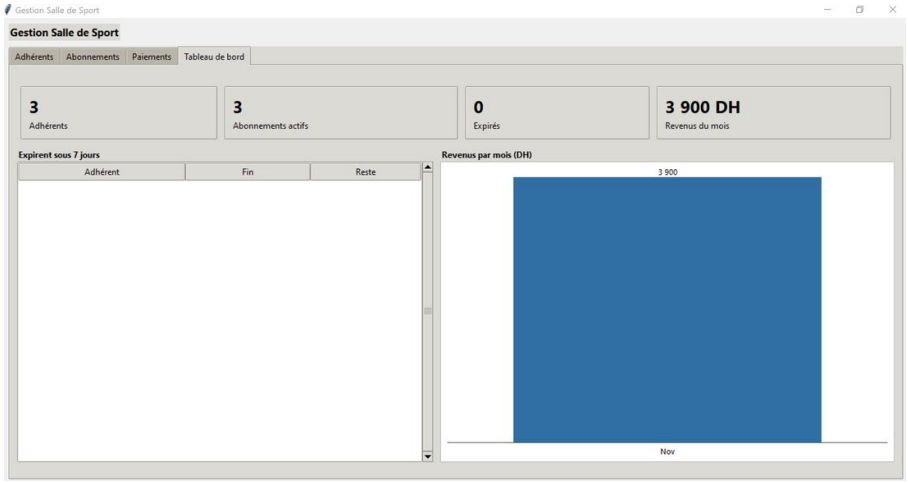


Figure 8 – tableau de bord : 3 adhérents, 3 abonnements actifs, revenus du mois de 3 900 DH.

```
TYPES = {"1 Mois": (1, 300), "3 Mois": (3, 800), "6 Mois": (6, 1500), "12 Mois": (12, 2800)}

def add_months(d, months):                  # fin = début + durée, fins de mois gérées
    m = d.month - 1 + months
    y = d.year + m // 12
    m = m % 12 + 1
    return date(y, m, min(d.day, calendar.monthrange(y, m)[1]))

def statut(date_fin_str):                   # texte + couleur de la ligne
    fin, ajd = date.fromisoformat(date_fin_str), date.today()
    if fin < ajd:                return "Expiré", "expire"
    elif (fin - ajd).days <= 7: return "Expire bientôt", "bientot"
    return "Actif", "actif"
```

5.2 Validation des saisies

Champ	Règle	Réaction
Nom, prénom	Non vides	Avertissement « obligatoires »
Téléphone	Facultatif ; chiffres uniquement (espaces et + tolérés)	Message « ne doit contenir que des chiffres »
Email	Facultatif ; contient @ et un domaine avec un point	Message « adresse email valide »
Date de début	Format AAAA-MM-JJ	Date de fin vide, création refusée
Prix	Nombre (virgule acceptée) > 0	Message « nombre positif »
Montant	Nombre (virgule acceptée) > 0 ; s'il dépasse le reste : confirmation	Erreur explicite ou boîte de confirmation (figure 9)
Suppressions	Adhérent, abonnement ou paiement sélectionné	Confirmation demandée ; cascade annoncée

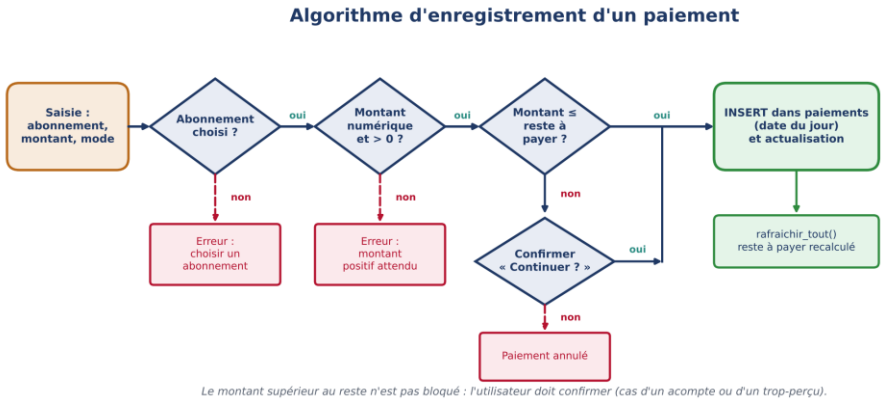


Figure 9 – contrôles successifs lors de l'enregistrement d'un paiement.

5.3 Données enregistrées dans la base

Pour vérifier la liaison entre l'interface et SQLite, nous avons ouvert gym.db avec DB Browser for SQLite. Les trois tables contiennent les mêmes données que l'application (identifiants, téléphones, emails, dates, montants) : 3 adhérents, 3 abonnements (1, 3 et 12 mois) et 3 paiements (300, 800 et 2 800 DH, soit 3 900 DH). L'adhérent n° 3, supprimé pendant les tests, est absent de la table adherents ; la table sqlite_sequence garde le dernier identifiant attribué à chaque table (AUTOINCREMENT).

Table : adherents

	id	nom	prenom	telephone	email	date_inscription
1	1	GURAINI	YAHYA	0650630259	guraini.yahya@ensam-casa.ma	2026-11-06
2	2	GAHEM	ISMAIL	0697992499	gahem.ismail@ensam-casa.ma	2026-11-06
3	4	QUASSAY	SAAD	0658151800	quassay.saad@ensam-casa.ma	2026-11-06

Table : abonnements

	id	*adherent_id*	type	date_debut	date_fin	prix
1	1	1	1 Mois	2026-11-06	2026-12-06	300.0
2	2	2	3 Mois	2026-11-06	2027-02-06	800.0
3	3	4	12 Mois	2026-11-06	2027-11-06	2800.0

Table : paiements

	id	*abonnement_id*	montant	date_paiement	mode
1	1	1	300.0	2026-11-06	Espèces
2	2	2	800.0	2026-11-06	Espèces
3	3	3	2800.0	2026-11-06	Espèces

Table : sqlite_sequence

	name	seq
1	adherents	4
2	abonnements	3
3	paiements	3

Figure 10 – contenu des tables adherents, abonnements, paiements et sqlite_sequence dans DB Browser.

6. Tests

Les tests ont été réalisés en pilotant l'interface par programme, sur une base temporaire (les boîtes de dialogue sont simulées) :

N°	Scénario	Résultat obtenu	Statut
1	Ajout de « Alaoui Sara », téléphone 0667000000	Enregistré ; le zéro initial du téléphone est conservé après sélection	✓Validé
2	Téléphone « 12ab », email « abc » ou nom vide	Refus avec le message d'avertissement correspondant ; rien n'est enregistré	✓Validé
3	Abonnement 3 Mois débutant le 2026-11-06	Fin proposée 2027-02-06 ; prix 800 DH	✓Validé
4	Début le 2026-08-31, type 1 Mois	Fin 2026-09-30 (fin de mois gérée)	✓Validé
5	Date de début invalide	Date de fin vide ; création refusée	✓Validé
6	Paiement de « 100,5 » sur un abonnement à 800 DH	Accepté ; déjà payé 100,5 DH ; reste à payer 699,5 DH	✓Validé
7	Paiement de 5000 DH (> reste), confirmation refusée	Aucun paiement enregistré	✓Validé
8	Montant « abc » ou « -5 »	Refus : « nombre positif »	✓Validé
9	Statut : fin hier, dans 3 jours, dans 60 jours	Expiré, Expire bientôt, Actif	✓Validé
10	Suppression de l'adhérent	Abonnements et paiements supprimés (cascade) ; compteurs à zéro	✓Validé

7. Difficultés rencontrées et solutions

Difficulté	Solution retenue
Onglets désynchronisés : une donnée créée dans un onglet n'apparaissait pas dans les autres.	Méthode rafraichir_tout() appelée après chaque opération et à chaque changement d'onglet.
Fins de mois : 31 août + 1 mois donne une date inexistante.	Fonction add_months() qui limite le jour au dernier jour du mois (monthrange).
Suppression en cascade sans effet : SQLite ignore les clés étrangères par défaut.	PRAGMA foreign_keys = ON à chaque connexion, avec ON DELETE CASCADE dans le schéma.
SUM() renvoie NULL (None) quand un abonnement n'a aucun paiement : le calcul du reste échouait.	Valeur par défaut 0.0 dans total_paye() et revenus_du_mois().
La sélection dans le tableau disparaissait à chaque actualisation.	rafraichir() rétablit la ligne sélectionnée si elle existe encore.

8. Conclusion et perspectives

L'application répond au cahier des charges : elle gère les adhérents, leurs abonnements et leurs paiements dans une base de trois tables cohérente, avec une interface claire, un tableau de bord et des contrôles de saisie. La séparation entre l'interface et la classe Database facilite la maintenance. Ce projet nous a permis de mettre en pratique la modélisation Merise, le SQL avec SQLite, la programmation d'interfaces avec Tkinter et le travail en équipe.

Améliorations possibles :

- renouveler un abonnement déjà expiré à partir de la date du jour (actuellement la prolongation part de l'ancienne date de fin) ;
- ajouter des contraintes dans la base (email unique, valeurs de type et de mode limitées, montant > 0) en plus des contrôles de l'interface ;
- exporter les données (CSV) et générer des reçus de paiement en PDF ;
- ajouter une authentification du gérant, des alertes d'expiration et une sauvegarde automatique de la base.

Merci pour votre lecture !